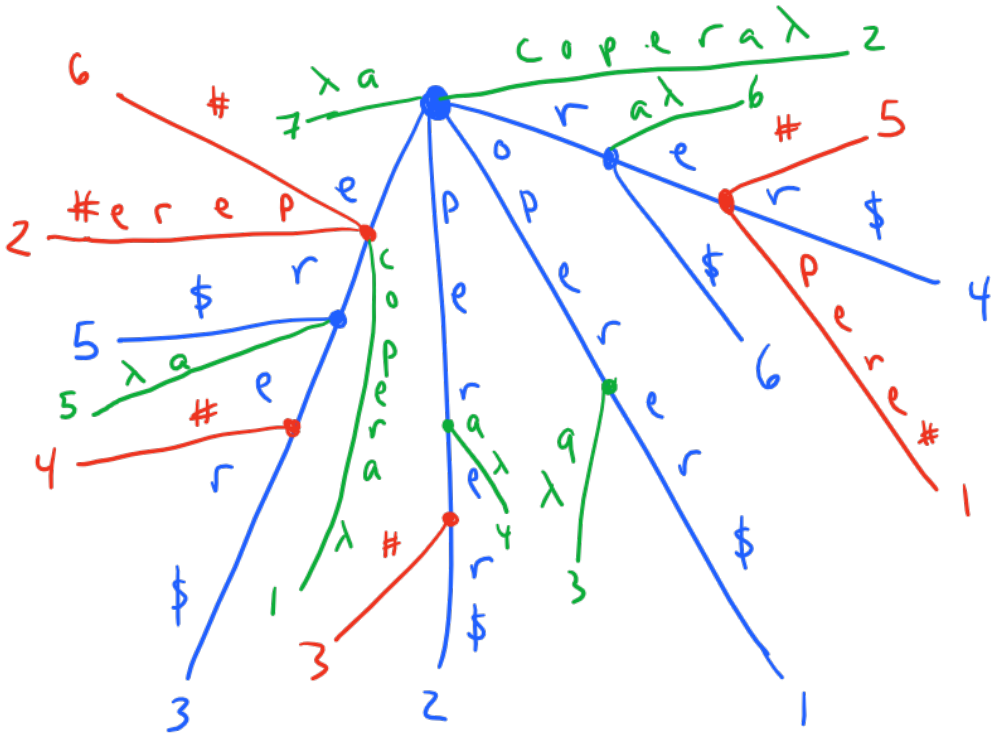
BIN702 - Série d'exercices #2 : les arbres de suffixe

Manuel Lafond

Exercice 1: Construisez l'arbre de suffixe généralisé pour les trois chaînes $operer, repere, ecopera$.

Solution.

On doit déterminer trois symbols de terminaison différents. Ici, j'utilise $\$, \#$ et λ.



Exercice 2: On a mentionné sans démonstration que puisqu'un arbre de suffixe A possède n feuilles et que chaque noeud interne a au moins 2 enfants, alors A possède $O(n)$ noeuds au total. En fait, on peut montrer que A a au maximum $2n-1$ noeuds et au maximum $2n-2$ arêtes. Pouvez-vous le démontrer ?

Solution.
Vous pouvez faire une démonstration formelle par induction sur le nombre de feuilles n. Comme cas de base, si $n = 1$, alors A possède $2n-1 = 1$ noeuds et $2n-2 = 0$ arêtes. Pour l'induction, soit A un arbre à n feuilles enraciné en un noeud r, et soient $r_1, \ldots, r_k$ les enfants de r. De plus, soient $n_1, \ldots, n_k$ les nombres de feuilles descendant de $r_1, \ldots, r_k$, respectivement. Par induction, ces arbres somment pour au plus $\sum_{i=1}^{k}(2n_i - 1)$ noeuds et au plus $\sum_{i=1}^{k}(2n_i - 2)$ arêtes. Puisque $\sum_{i=1}^{k} n_i = n$, le nombre de noeuds est donc au plus, en comptant r, $1 + \left(\sum_{i=1}^{k} 2n_i\right) - \sum_{i=1}^{k} 1 = 1 + 2n - k \leq 2n - 1$ car $k \geq 2$. De la même façon, le nombre d'arêtes est au plus $2n - 2k + k \leq 2n - 2$ car $k \geq 2$.

Exercice 3: Il est possible que deux séquences S et T aient plusieurs sous-chaînes communes de taille maximum. Donnez un algorithme qui retourne le **nombre** de plus longues sous-chaînes communes entre S et T. Quelle est la meilleure complexité que vous pouvez atteindre ?

Solution.
Ceci peut se réaliser en suivant les étapes suivantes.

a. Construire l'arbre de suffixe généralisé A pour S et T.
b. Parcourir A de bas en haut. À chaque noeud v, retenir si v a une feuille descendante de S et/ou T.
c. Trouver la profondeur en caractère maximum p d'un noeud ayant une feuille descendante de S *et* de T.
d. Parcourir l'arbre pour compter le nombre de noeuds qui ont une profondeur p ayant une feuille descendante de S et T.

Ceci compte le nombre de chaînes distinctes de longueur p qui sont présentes dans S et T. On aurait pu intrepréter la question comme demandant le nombre de paires de positions où démarrent des sous-chaînes communes. Dans ce cas, il faudrait, pour chaque noeud v trouvé à la dernière étape, retourner le nombre de feuilles descendantes à v dans S, fois le nombre de feuilles descendantes à v dans T. J'accepterais les deux types de réponses car la question était un peu ambigue.

Chaque étape peut se faire en temps linéaire, le temps est donc $O(n)$. Ci-bas, je vous propose un pseudo-code qui fait ces 4 tâches à la fois. C'est un peu laborieux, mais l'idée est que chaque noeud retourne à son parent s'il a des feuilles de S ($hasS$), des feuilles de T ($hasT$) et, s'il en a des deux, retourne la profondeur maximum d'un de ses descendants ayant les deux et le nombre de tels descendants.
Ceci serait une réponse acceptable en examen. On voit assez bien les étapes importantes.
Pour vos fins d'apprentissage, je laisse tout de même ma version pseudo-code de la chose. Si r est la racine de l'arbre, il faudrait faire un appel initial avec $nbCommonSubstr(r, 0)$.

```
fonction nbCommonSubstr(u, curprof)
    //Retourne (maxProf, nbCommun, hasS, hasT);
    si u est une feuille alors
        si u est une position de S alors
            return (0, 0, True, False);
        sinon
            return (0, 0, False, True);
    fin
    curMaxProf = 0;
    curNbCommun = 0;
    hasS = False;
    hasT = False;
    pour chaque enfant v de u faire
        //uv.longueur est le nombre de caractères sur l'arête uv;
        (vmaxProf, vcommun, vhasS, vhasT) =
         nbCommonSubstr(v, curProf + uv.longueur);
        si vhasS et vhasT alors
            si vmaxProf > curMaxProf alors
                curMaxProf = vmaxProf;
                curNbCommun = vcommun;
            sinon si vmaxProf = curMaxProf alors
                curNbCommun = curNbCommun + vcommun;
        fin
        hasS = hasS ∨ vhasS ;          // ∨ est le ''ou'' logique
        hasT = hasT ∨ vhasT;
    fin
    si curMaxProf > 0 alors
        return (curMaxProf, curNbCommun, True, True);
    sinon si hasS et hasT alors
        //On entre ici quand v est le premier ayant S et T;
        return (curProf, 1, True, True);
    sinon
        return (curProf, 0, hasS, hasT);
```

Exercice 4: Montrez comment calculer la plus longue sous-chaîne commune entre k séquences $S_1, S_2, \ldots, S_k$.
Version facile : en temps $O(kn)$. Pas mal plus difficile : $O(n)$.

Solution.

Dans la version facile, on construit l'arbre de suffixe généralisé pour $S_1, \ldots, S_k$, puis de bas en haut, on étiquette chaque noeud interne avec la liste des S_i qui sont représentés parmi les descendants. Cette étiquettage peut se faire avec un parcours post-order : l'étiquette est l'union des étiquettes des deux enfants. La plus longue chaîne commune est celle de profondeur maximum contenant tous les S_i comme étiquette. Puisque chaque noeud interne reçoit $O(k)$ étiquettes et qu'il y a $O(n)$ noeuds, ceci prend un temps $O(nk)$.
La version difficile est longue à expliquer. Je vous renvoie à `https://web.cs.ucdavis.edu/~gusfield/cs224f09/commonsubstrings.pdf`, car je ne pourrais pas l'expliquer d'une meilleure façon.

Exercice 5: Supposez que vous avez un tableau de suffixe $SA(T)$ pour une séquence T de taille n. Supposez aussi que pour chaque position $i < n$, vous avez accès en temps $O(1)$ au plus long préfixe commun (PLPC) entre le suffixe représenté par $SA(T)[i]$ et le suffixe représenté par $SA(T)[i+1]$.
Montrez comment reconstruire l'arbre de suffixe en temps $O(n)$.

Solution.
Rappelons que dans le tableau de suffixes, les positions de T sont triées selon l'ordre lexicographique des suffixes. L'idée est de construire l'arbre en introduisant les suffixes en ordre lexicographique. Chaque fois qu'on insère un suffixe commençant à la position $SA(T)[i]$, ce suffixe sur l'arbre est formé du chemin qui commence à la racine, et qui suit le chemin du suffixe $SA[i-1]$ jusqu'à profondeur $PLPC(SA(T)[i-1], SA(T)[i])$ (ceci est la clé, prenez le temps de comprendre pourquoi, si ce n'est pas déjà fait). Rappelons que $PLPC(i, j)$ est la longueur du plus long préfixe commun entre $S[i..n]$ et $S[j..n]$.
Une fois arrivé à la profondeur $PLPC(SA(T)[i-1], SA(T)[i])$, le prochain caractère du suffixe en $SA[i]$ diffère de celui en $SA[i-1]$, et on ajoute une

nouvelle branche pour le reste du suffixe de $SA[i]$.

fonction *SaToSt(SA, T)*
- $A =$ arbre avec un seul noeud racine r;
- Ajouter une branche à A pour le suffixe commençant à $SA[1]$;
- $f =$ la feuille ajoutée à l'instruction précédente;
- **pour** $i = 2..n$ **faire**
  - $\ell = PLPC(SA[i], SA[i-1])$;
  - En démarrant de f et en remontant l'arbre, trouver l'arête e contenant le caractère du suffixe en $SA[i-1]$ de profondeur ℓ;
  - Ajouter une arête sortant de la branche e, étiquetée par la longueur de $SA[i]$ moins ℓ, avec une feuille $SA[i]$;
  - Soit f cette feuille qui a été ajoutée;
- **fin**

Cet algorithme prend un temps linéaire, en particulier parce que lorsqu'on cherche la branche e voulue, on remonte à partir d'une feuille f qui n'est jamais la même entre deux itérations. Donc chaque branche est explorée au plus une fois dans les étapes de "remontée". Je vous laisse réfléchir aux détails.

Exercice 6: Rappelons qu'un palindrome-complément est une séquence S de taille $2n$ telle que $S[1..n]$ est égale à $S[n+1..2n]$ lue de droite à gauche après avoir fait les échanges de caractères $A \leftrightarrow T$ et $C \leftrightarrow G$. Par exemple, $AACTGCAGTT$ est un palindrome-complément car on a $[AACTG]$ dans la première moitié et, en complémentant puis inversant l'autre moitié, on a $[CAGTT] \rightarrow [GTCAA] \rightarrow [AACTG]$.
Montrez comment trouver tous les palindrome-compléments maximaux dans une séquence T.

Solution.
Notons qu'un palindrome-complément est nécessairement pair. Il suffit de complémenter S et de la renverser, obtenant ainsi une séquence qu'on va appeler S'. Ensuite, pour chaque position i, on veut le plus long palindrome dont la deuxième moitié commence en i. Celui-ci est le plus long préfixe commun entre $S[i..n]$ et $S'[n-i+2..n]$.
Par exemple, soit $S = TTAGCTT$, on a le palindrome-complément $[AGCT]$. On a $S' = AAGCTAA$. Quand $i = 5$, on a $n - i + 2 = 4$, $PLPC(5, 4) = 2$ (visuellement parce que $S = TTAG\underline{CT}T$ et $S' = AAG\underline{CT}AA$). Le CT souligné dans S' correspond au AG de S, indiquant qu'il y a un palindrome-complément dont la 2ème moitié est de longueur 2 et commence à CT.

Exercice 7: La distance Hamming entre deux séquences S et T de même longueur est le nombre de positions auxquelles S et T diffèrent. Par exemple, $S = ACCT$ et $T = AGCT$ ont une distance Hamming de 1.
Soient P un mot à chercher et T une séquence plus longue. Montrez comment on peut trouver toutes les sous-chaînes T' de T de longueur $|P|$ telles que T' et P sont à distance Hamming de 0 ou 1. (on peut le faire en temps $O(n + occ)$)
De façon plus générale, montrez comment trouvez toutes les sous-chaînes de T à distance Hamming au plus k de P, et ce en temps $O(kn)$.
(indice : il y a une façon qui ne fait qu'utiliser des appels à PLPC(i, j))

Solution.
Soient $n = |S|$ et $m = |T|$. On définit $PLPC(i, j)$ comme la longueur du plus long préfixe commun entre $S[i..n]$ et $T[j..m]$.
Supposons $k = 1$. L'idée est d'essayer de trouver une occurrence de T à distance Hamming de 1 à chaque position de i de S. S'il y a une telle occurrence, on sait qu'on va matcher $j = PLPC(i, 1)$ caractères, que le caractère non-matché sera à la position $i + j + 1$ de S (et $j + 1$ de T), et qu'on devra avoir $PLPC(i + j + 2, j + 2) = m - j$ (c'est-à-dire, on doit matcher le reste de T).
En d'autres termes, on fait des bonds de PLPC. Pour k en général, on a le droit de faire k bonds, et il faut qu'on réussisse à épuiser tous les caractères de T. Vous pouvez soit regarder le pseudo-code ci-bas, ou bien consulter `http://www.cs.tau.ac.il/~haimk/adv-alg-2014/pattern-matching-with-mismatches.pdf` qui l'explique de façon plus jolie.

```
fonction recherchelnexacte(S, T, k)
    Construire l'arbre de suffixe généralisé pour S et T;
    Prétraiter l'arbre pour répondre à des requêtes de LCA en temps
      O(1);
    pour i = 1..n faire
        j = 0 // Nombre de caractères de T épuisés
        ℓ = 0 // Nombre de mismatches
        tant que ℓ < k et j ≤ m faire
            j = PLPC(i + j, 1 + j) + 1;
            ℓ = ℓ + 1;
        fin
        si j > m alors
            Rapporter une occurrence en i;
        fin
    fin
```

Sachant que chaque PLPC prend un temps $O(1)$ en utilisant le LCA, ceci

prend un temps $O(nk)$.